

YOLO 卷积函数的 SMART 仿真

2021 年 5 月

一、实验介绍

实验 5 介绍了 YOLO 算法的基本原理以及其 C 源码的代码框架，并在基于 RISC-V QEMU 的 Linux 操作系统上验证了该算法的前向传播功能。实验 6 至实验 8 则介绍了 C910 SMART 平台的配置和仿真方法。本实验将在上述过程的基础上，将 YOLO 源码的卷积函数放在 SMART 上进行仿真和性能监测，从而进一步介绍嵌入式 C 裸机程序在 C910 上的开发过程和仿真原理。

二、C 程序在 SMART 的仿真流程

前面的实验对 SMART 提供的测试样例进行了仿真并讲解了 C910 核的配置方法，但并未对一个嵌入式程序在平台上仿真所经历的全部流程进行仔细的说明。若使用文本编辑软件打开平台目录下的./tools/run_case 脚本，则可以看到仿真过程实际上经历了多个步骤，本实验将结合 YOLO 卷积层的卷积函数，介绍 SMART 对 C 程序的仿真原理和注意事项，并借此讲述平台 RTL 模型的一些细节。仿真用到的卷积函数相关源码存放在文件夹/home/ECDesign/ECDesign_share/lab9/conv_test/下。

2.1 源码编译和链接

本节开始均假设工作路径在 smart9_release/下。平台的./tools/run_case 脚本解析命令行参数获取源码的路径后，会使用./lib 下的 Makefile 对源码进行编译和链接，使用的 RISC-V 工具链存放在目录./tools/toolchain/RV64GC 下，这是一个由平头哥定制的工具链。同时./lib 下还包含两个重要的文件，即**启动代码 crt0.s**和**链接脚本 linker.lcf**。如实验 2-3 所说，在操作系统上运行 C 程序时，用户通常不需要过多关注这个过程，因为操作系统完成了绝大部分的工作。但对裸机嵌入式而言，程序的启动和加载位置至关重要。首先查看./lib/linker.lcf，可以看到 SMART 使用的链接脚本对内存的划分规则为（下文省略了 SECTIONS 后半部分的内容）：

```
MEMORY
{
MEM1(RWX)  : ORIGIN = 0x00000000,  LENGTH = 0x40000
MEM2(RWX)  : ORIGIN = 0x00040000,  LENGTH = 0xc0000
}
__kernel_stack = 0xee000 ;
ENTRY(__start)
SECTIONS {
    .text :
```

```

{
    crt0.o (.text)
    *(.text*)
} >MEM1
//后略，完整内容可自行查看 linker.lcf 文件
//...
end = .;
}

```

可以看出平台默认把需要加载程序数据的内存组织为两段，0x0~0x40000 为 MEM1，存放代码段(.text)和只读数据段(.rodata 等)，而 0x40000~0x100000(即 0x40000+0xc0000)为 MEM2，存放其他已初始化和未初始化的数据(.data .bss 和.COMMON 等)。可见 Smart 平台使用了哈佛架构。

在平台仿真时，CPU 将默认从内存地址 0x0 开始读取和执行指令，而从链接脚本可以看出将被存放在起始地址的正是 crt0.o 中代码段的内容，这也是该文件被称为“启动代码”的原因。lib 目录下的 crt0.s 由汇编风格写成，用户可以自行阅读，总体上该文件执行以下过程：

- a) 对各种硬件寄存器、栈指针和控制信息的初始化。在 cache 和 pipeline 实验中，PMU 性能监测模块所需要添加的寄存器初始化也是在这个部分进行的。
- b) 完成初始化后，CPU 已经进入正确的状态，此时执行一条跳转指令：

```
jal main
```

从而跳转到用户编写的 C 程序主函数入口。

- c) main 函数执行完毕或遭遇错误后，视程序的执行结果，控制将返回 crt0 文件中定义的 _exit 函数或 _fail 函数中，并将某个通用寄存器设置为一个特殊数值，而仿真文件 tb.v 在检测到这个特殊数值后，将打印信息并结束仿真。

综上所述，在编译和链接过程中，GCC 工具链依据链接脚本对段存储位置的划分，生成完全链接的二进制文件（.elf 文件），只要这些指令和数据被放在与 linker.lcf 定义相一致的内存位置，CPU 就可以顺次取指并完成程序的执行。

2.2 生成便于 RTL 仿真模型加载的文件

在生成.elf 二进制文件后，Makefile 中还进行了以下操作：

```

INST_HEX = ${FILE}_inst.hex
DATA_HEX = ${FILE}_data.hex

${FILE}.hex : ${FILE}.elf
    ${OBJCOPY} ${HEXFLAGS} $< ${INST_HEX} -j .text* -j .rodata*
    ${OBJCOPY} ${HEXFLAGS} $< ${DATA_HEX} -j .data* -j .bss*
    ${OBJCOPY} ${HEXFLAGS} $< $@ |

INST_PAT = inst.pat
DATA_PAT = data.pat

%.pat : %.hex
    rm -f *.pat
    ${CONVERT} ${INST_HEX} ${INST_PAT}
    ${CONVERT} ${DATA_HEX} ${DATA_PAT}

```

上述过程生成两种文件：.hex 和.pat，首先 RISC-V 工具链中的 objcopy 工具会将.elf 文件转换成.hex 文件，这是一个 ASCII 文件，可供用户直接阅读，该文件以 16 进制字符直观地展示了最终载入内存的所有内容。

.hex 文件中包含了.text 段、.data 段等所有内容，为了便于 SMART 平台的仿真脚本./tb/tb.v 分别载入这些内容，Makefile 紧接着使用 SMART 提供的脚本./tools/Srec2vmem 将.hex 文件拆分为两个 ASCII 文件 inst.pat 和 data.pat，两个文件存放的数据分别对应上述链接脚本中的 MEM1 和 MEM2 两段。以 inst.pat 为例，该文件的组织形式为：

```

@00000000  b7014000  73a0017c  996173a0  0130b701
@00000004  800073a0  0130a161  73a0017c  81400141
@00000008  81410142  81420143  81430144  81440145
@0000000c  81450146  81460147  81470148  81480149
@00000010  8149014a  814a014b  814b014c  814c014d
@00000014  814d014e  814e014f  814ff321  40f10542
@00000018  17e10e00  130101fa  97010000  93816112

```

文件每行存放 16 个字节的数据（128 位），每 4 个字节（32 位）为一组，每行 4 组，**小端存放**。开头的@编号为数组索引。inst.pat 和 data.pat 的内容符合 Verilog 中系统任务 \$readmemh 的格式要求，因此可以直接被 testbench 读入。

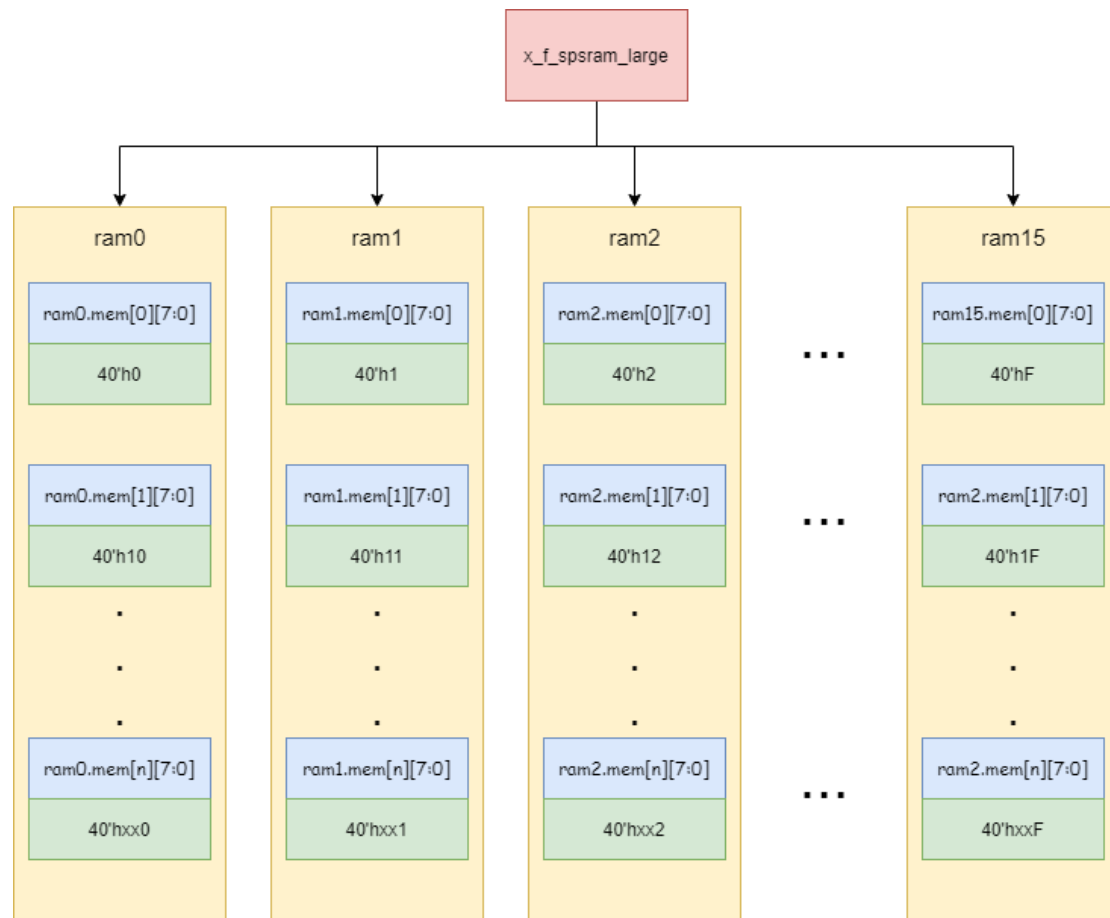
2.3 加载指令和数据到仿真模型

应当注意，与在 FPGA 板上运行嵌入式程序不同，SMART 是一个使用 irun 或 vcs 进行 RTL 仿真的平台，所提供的 CPU 核、总线、Memory 及其他外围模块均是 RTL 模型而非真实硬件。因此程序数据是通过 testbench 初始化到某块被例化的 Memory RTL 模型上去的。具体而言，在./tb/tb.v 中可以找到如下定义：

```
`define RTL_MEM          tb.x_soc.x_axi_slave128.x_f_spsram_large
```

依照该层次索引到的模块 x_f_spsram_large 即为 Verilog 中实例化的内存模型，该内存模块的地址映射由其上层模块 x_axi_slave128 完成，对于本例而言，映射后的起始地址就是 0x0。

x_f_spsram_large 对应的内存模型结构如下：



每行由 ram0~ram15 构成 16 个字节，40 位 pc 当中的 pc[32:4]传入 x_f_spsram_large 进行寻址，因此该 Memory 的理论最大深度为 2^{29} ，模块总是传出一整行的 16 字节数据，而 40 位 pc 的最低 4 位 pc[3:0]则可对行内 16 字节的每个字节进行寻址，从而该模型可以取出定义范围内的任意字节。

基于上述内存模型，./tb/tb.v 首先把 inst.pat 和 data.pat 中的内容用系统任务\$readmemh 读入到一个中间数组中：

```
$readmemh("inst.pat", mem_inst_temp);
```

```
$readmemh("data.pat", mem_data_temp);
```

然后通过一个 initial 过程直接对内存模型进行初始化：

```
i=0;
for(j=0;i<32'h4000;i=j/4)
begin
  `RTL_MEM.ram0.mem[i][7:0] = mem_inst_temp[j][31:24];
  `RTL_MEM.ram1.mem[i][7:0] = mem_inst_temp[j][23:16];
  `RTL_MEM.ram2.mem[i][7:0] = mem_inst_temp[j][15: 8];
  `RTL_MEM.ram3.mem[i][7:0] = mem_inst_temp[j][ 7: 0];
  j = j+1;
  `RTL_MEM.ram4.mem[i][7:0] = mem_inst_temp[j][31:24];
  //后略，详细代码参见 tb/tb.v
```

```

end
i=0;
for(j=0;i<32'h4000;i=j/4)
begin
  `RTL_MEM.ram0.mem[i+32'h4000][7:0] = mem_data_temp[j][31:24];
  `RTL_MEM.ram1.mem[i+32'h4000][7:0] = mem_data_temp[j][23:16];
  `RTL_MEM.ram2.mem[i+32'h4000][7:0] = mem_data_temp[j][15: 8];
  `RTL_MEM.ram3.mem[i+32'h4000][7:0] = mem_data_temp[j][ 7: 0];
  j = j+1;
  `RTL_MEM.ram4.mem[i+32'h4000][7:0] = mem_data_temp[j][31:24];
  //后略，详细代码参见 tb/tb.v
end

```

注意到 inst.pat 和 data.pat 每行存储了 16 个字节信息, 这恰与内存模型的一行 ram0~ram15 相对应。除此之外, 与内存结构图相比较即可发现, 在第二个循环中内存的起始位置 `RTL_MEM.ram0.mem[32'h4000]恰好就是地址 0x40000 (内存的每一行包含 16 个字节, 因此第 0x4000 行的 PC 值为 $0x4000 \times 16 = 0x40000$), 这正是 linker.lcf 中所定义的 MEM2 (对应信息存放于 data.pat) 的起始地址。因此, testbench 就通过这样一种简单直接的方式, 把指令编码和程序数据初始化到了正确的内存位置。

完成上述过程后, CPU 的 RTL 模型通过其内部控制逻辑, 从 0x0 位置开始取指并顺次执行, 直到触发仿真结束条件。自此, 一个嵌入式 C 程序在 SMART 平台上从编译、链接、初始化至内存模型到最终仿真的全过程就执行完毕。

三、 实现输入和输出

嵌入式系统通常不直接支持标准输入输出函数 scanf 和 printf, 一般需要用户重写相关函数的实现, 利用串口等通信方式来完成数据的输入和输出。而对于 SMART 平台, 由于我们可以在仿真模型中接触到硬件底层, 故输入和输出可以通过一些更简单直接的方式实现。

3.1 标准输出函数 printf

printf 最终将会调用 write 函数来实现字符输出, 该函数的原型为:

```
size_t write(int fd, const void* buf, size_t count);
```

其中, fd 为指示输出目的地的文件描述符, buf 为需要写出的字符串首指针, 而 count 则为要写出的字节数。对于 Linux 的文件系统而言, 默认会有三个文件被打开, 即/dev/stdin、/dev/stdout、/dev/stderr, 对应的文件描述符分别为 0/1/2。

SMART 重写了桩函数 write 以实现仿真平台上的输出, 代码位于 ./lib/clib/write.c, 编译时它会替代原有的 write 函数, 其实现如下:

```
#define write_char(x) asm("li x13, 0x0003fff8; sw %0, 0(x13)" : : "r" (x))
int _write (int fd, void *buf, size_t count)
{
    uint8_t * buf_tmp = (uint8_t *)buf;
    write_char(buf_tmp[0]);
}
```

桩函数调用了一条内联汇编指令，把要打印的字符写到地址 0x3fff8 上面，然后只需要在 testbench 里面不断检测 CPU 的地址线（即`SOC\_TOP.x\_axi\_slave128.mem\_addr[31:0]`），当该地址线的值等于 0x3fff8 时即表明发生了_write 桩函数的调用，此时直接把仿真模型中总线上的数据信息（即`SOC\_TOP.biu\_pad\_wdata[127:0]`）取出，并调用 Verilog 的系统任务\$write 打印该字符即可。

该实现具有某些安全性方面的问题，但胜在简单。原则上 0x3fff8 这个地址的指定是比较随意的，只要是一个嵌入式程序不会占用的空闲地址，且保证桩函数和 testbench 监测的地址相匹配即可。此外，该实现并不返回状态信息，且每次通过 Verilog 的\$write 任务打印一个字符。**在使用过程中建议每次调用 printf 函数时均在字符串末尾加上换行符“\n”，否则可能会出现缓冲区被覆盖而无法正确输出的情况。**

在本实验后续的性能统计中可以看到，printf 函数会占用很多的时钟周期才能完成，频繁地调用该函数会大幅增加仿真时间，因此**使用 SMART 平台进行仿真时不建议在源码中频繁使用 printf 函数。**

3.2 输入数据的初始化

YOLOv3 中的第一个卷积层输入为 1108992 个单精度浮点数，数据规模约为 4MB，如何对这样的输入变量进行初始化也是开发中需要考虑的问题。至少有两种解决方案：

- 预先要把要初始化的数据显式地存放在源码中（例如写在一个.h 头文件里），然后直接在代码内对变量赋值，这样就避免了数据从外部读入的问题。但一方面，该方案使代码变得臃肿难读；另一方面，**编译器会自动把源码中初始化的数据分配到数据段中**，如前文所述，链接脚本划分的空间长度是有限的。以本实验为例，linker.lcf 中储存数据段的 MEM2 长度为 0xc0000，折合 768KB，是远远无法放下卷积层的单层输入的。如果要使用该方式，需要额外修改 linker.lcf 中的定义。
- 第二种方案借鉴了前文中 testbench 装载机器码到内存模型的思路，在仿真阶段把要初始化变量的值通过 initial 过程加载到内存 RTL 模型中的特定位置，记录下该内存的起始地址，然后在 C 源码中直接为这些变量指定该地址，即可将变量名和它们在内存中的值关联起来。

为此需要对 SMART 上的总线模型做一些调整，分别查阅文件 ./rtl/platform/common/soc.v 和文件 ./rtl/platform/amba/axi/axi_interconnect128.v 可以得到 SMART 平台对总线模块起止地址的定义如下：

总线模块名 (定义于 soc.v)	起始地址 (定义于 axi_interconnect128.v)	结束地址 (定义于 axi_interconnect128.v)
x_axi_slave128	SRAM_START = 40'h0000_0000	SRAM_END = 40'h01ff_ffff
x_axi_err	ERR1_START = 40'h0200_0000	ERR1_END = 40'h0fff_ffff
x_axi2ahb	APB_START = 40'h1000_0000	APB_END = 40'h1fff_ffff
x_axi_err1	ERR2_START = 40'h2000_0000	ERR2_END = 40'hfff_ffff_ffff

这个划分表明当前总线上挂载了 4 个不同的 AXI 模块，axi_interconnect128 模块的作用是接受 CPU 的读写地址作为输入，模块内部依照上述地址段的划分产生选通信号作为输出，以决定总线上这 4 个模块中的哪一个会被选通，同一时间仅有一个模块被选通。对于链接脚本 linker.lcf 所划分的 MEM1、MEM2 和栈空间，其地址全部落在 SRAM_START 和 SRAM_END 之间，因此总是操作 x_axi_slave128 这一模块内的内存模型。

仿真模型中其实只有第一段 x_axi_slave128 被实际使用了，其余的三个模块虽然也被集成到 soc.v 中，但仅起到占位作用。因此可以把 soc.v 当中的 x_axi_err 模块替换成用户重写的具有正常功能的 AXI 模块，并保证其内部的内存模型基地址被映射到 0x0200_0000（即 ERR1_START），这样就开辟出了一块可以被 CPU 正确访问的新物理地址空间。

为此，需要将新文件 axi_slave128_copy.v 复制到目录 ./rtl/platform/amba/axi/ 下，并用本实验文件夹中提供的 soc.v 替换掉原有的 ./rtl/platform/common/soc.v 文件。axi_slave128_copy.v 复用了 axi_slave128.v 的几乎全部代码，区别仅在于后者将 memory 起始位置映射到 SRAM_START，而前者则映射到 ERR1_START。

完成上述替换后，在 ./tb/tb.v 作如下定义：

```
`define RTL_MEM2      tb.x_soc.x_axi_slave128_copy.x_f_spsram_large
```

然后使用编写的脚本把要初始化的输入数据组织成与 inst.pat 和 data.pat 相仿的形式，并命名为 data_in，用和 RTL_MEM1 相同的方式进行赋值：

```
$readmemh("data_in",mem_datain_tmp);
j=0;
for(i=0;i<32'h43BD80;i=i+16)
begin
`RTL_MEM2.ram0.mem[j][7:0]= mem_datain_tmp[i][7:0];
`RTL_MEM2.ram1.mem[j][7:0]= mem_datain_tmp[i+1][7:0];
`RTL_MEM2.ram2.mem[j][7:0]= mem_datain_tmp[i+2][7:0];
//后略，参见./tb/tb.v
//...
`RTL_MEM2.ram15.mem[j][7:0]= mem_datain_tmp[i+15][7:0];
j= j + 1;
end*/
```

现在数据被初始化到了以 0x0200_0000 开始的内存模型中，还需要完成的就是在 C 源码中为变量指定地址，对本实验所使用的 YOLO 卷积函数而言，代码为：

```
#define DATA_BASE_ADDR (0x02000000)

volatile float *input = DATA_BASE_ADDR;
volatile float *weights = input + 1.inputs;
volatile float *output = weights + 1.nweights;
volatile float *workspace = output + 1.outputs;
l.output = output;
l.weights = weights;
net.worksapce = workspace;
net.input = input;
```

通过直接为 C 变量指定地址，就自动完成了变量名与值的关联。

由于 YOLOv3 单个卷积层的输入输出规模较大，在 SMART 上面的仿真时间非常长，因此**本实验将使用一个较小规模的输入进行仿真**，但仍然会采用上述第二种方案进行数据的读入。经过处理的输入数据文本 data_in 已经放置在本实验的文件夹下，用户可以直接使用。

四、YOLO 卷积函数仿真的性能监测

基于上述改动，已经可以对 ./case/conv_test/conv_test.c 进行仿真，这是一个小规模输入的卷积函数。在 ./workdir 目录下运行以下命令即可进行仿真：

```
../tools/run_case ../case/conv_test/conv_test.c
```

本实验要求对该程序的仿真过程进行三个方面的性能监测。

4.1 CPI、cache 缺失率和分支预测统计

统计原理与实验 7（cache 操作实验）类似，可以通过 C910 的性能监测单元 PMU 完成，为此应当在启动代码 crt0.s 中初始化相关的 CSR，并且把相关 C 函数的封装 pmu.h 添加到编译路径下。

本实验文件下的源码 conv_test.c 当中已经加入了相关的统计函数，直接运行即可。

由于后文指令分布统计的代码仅针对单核的情况编写，因此务必确认配置 CPU 核时“Core Num”选项的设置为 1，即仅生成一个核心。否则，用户应当根据实验 7 提供的操作方法，使用 C910_R1S2P19/文件夹下的配置工具重新生成一个单核的 CPU。

不同 Cache 的大小对本实验统计结果的影响微乎其微（可以自行思考原因），因此可以使用任意大小的 Cache 配置进行实验。

4.2 指令分布的统计

该过程旨在对卷积层的卷积计算所调用的机器指令按照类型进行统计，并画出各指令类型执行次数的分布图。性能监测单元 PMU 并不直接提供相关功能，因此需对 testbench 进行额外的改动来自行实现这个功能。下面介绍我们使用的方法。

C910 集成了以下观测信号在 CPU 顶层以供使用（详见实验 6 文档附录 1）：

CPU 运行观测信号集成:				
core(x)_pad_retire0	O	1'b0	CPU	各个处理器核指令 0 退休指示信号: 0: 当前周期没有指令退休; 1: 当前周期有指令退休。
core(x)_pad_retire0_iid[6:0]	O	7'b-	CPU	各个处理器退休指令 0 的指令 ID: 标明当前正在退休指令的 ID。
core(x)_pad_retire0_pc[39:0]	O	-	CPU	各个处理器核退休指令 0 的 PC: 表明当前正在退休的指令的 PC。

C910 为三发射超标量处理器，每个核心拥有三个指令退休信号 core(x)_pad_retire0、core(x)_pad_retire1、core(x)_pad_retire2，依次对应三个发射单元。它们三者中的任意一个为高时即表明当前周期有指令将要退休。此时相应的 core(x)_pad_retire0_pc[39:0]、core(x)_pad_retire1_pc[39:0]或 core(x)_pad_retire2_pc[39:0]上即存储了正在退休的指令 PC，该 PC 对应内存位置储存的值正是这条退休指令的机器编码，因此只要解析机器码的 opcode，就可以对指令进行分类和计数。

所幸对于仿真模型而言，要取得 PC 对应位置的机器码并不需要真的对内存位置进行访问，在 testbench 的初始化阶段，已经在一个 initial 过程中通过 \$readmemh 任务把 inst.pat 中的内容读入到了一个寄存器数组 mem_inst_temp 当中，故只要使用 PC 对这个寄存器数组进行寻址就可以取出 opcode。

在此基础上，还有两点细节需要关注：

1. 如何区分指令的长度

在 RISC-V64GC 指令集中，指令可以是 32 位的（对应于 G），也可以是经过压缩后的 16 位指令（对应于 C），且 16 位指令可以任意地穿插于 32 位指令之间而不需要经过任何额外的对齐操作。统计时需要识别当前退休的指令是 4 字节还是 2 字节。

查阅 RISC-V 的 Specification 文件知，可以通过指令最低 2 位判断指令类型属于 G 还是 C，32 位指令总是使用 2'b11 作为最低位，且最低 7 位为 opcode。以下图为例：

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0	
funct7				rs2		rs1	funct3		rd			opcode		R-type	
imm[11:0]						rs1	funct3		rd			opcode		I-type	
imm[11:5]				rs2		rs1	funct3		imm[4:0]			opcode		S-type	
imm[12]	imm[10:5]			rs2		rs1	funct3		imm[4:1]	imm[11]	opcode		B-type		
imm[31:12]									rd			opcode		U-type	
imm[20]	imm[10:1]			imm[11]		imm[19:12]			rd			opcode		J-type	

而 16 位指令的最低两位则可以是 2'b00、2'b01、2'b10，同时结合最高 3 位编码指明具体指令类型，以下图为例：

Format	Meaning	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CR	Register	funct4				rd/rs1				rs2				op			
CI	Immediate	funct3		imm		rd/rs1				imm				op			
CSS	Stack-relative Store	funct3		imm				rs2				op					
CIW		funct3		imm								rd'		op			
CL	Load	funct3		imm				rs1'		imm		rd'		op			
CS	Store	funct3		imm				rs1'		imm		rs2'		op			
CA	Arithmetic	funct6				rd'/rs1'				funct2		rs2'		op			
CB	Branch	funct3		offset				rs1'		offset				op			
CJ	Jump	funct3		jump target												op	

因此在 testbench 中得到正在退休指令的 PC 后，首先索引到 PC 对应的字节取出最低 2 位（由于是小端存储，所以最低位总是位于 PC 对应的第一个字节，无论指令是 4 字节还是 2 字节），如果为 2'b11，说明是 32 位指令，取该字节最低 7 位作为 opcode 作为分类依据。如果不为 2'b11，说明是 16 位指令，则同时取指令的最高 3 位和最低 2 位一并作为分类依据。

2. 如何确定计数时机

指令类型按照 opcode 在 testbench 中进行计数，应当考虑计数的起止时间如何选定。一种简单的方法是在仿真之初就开始计数，直到程序退出结束计数并打印结果。该方法的潜在问题是，源文件 work.c 当中并不仅仅包含卷积层的传播函数，还包含了多个输出函数 printf。但 **printf 是一个会大量占用指令周期的函数**，如果把这部分调用所涉及的退休指令也纳入类型统计，将会导致统计结果失真，无法真实反应卷积函数所涉及的指令分布。

为此在源码中添加两个哨兵函数 guard_start 和 guard_end，这两个函数本身并无实质性的操作。此后，把源码设计成如下形式：

```
guard_start();
{
    // Test Code Here
}
gurad_end();
```

将想要统计的代码夹在 gurad_start 和 guard_end 中间，代码经过编译和重定向后，在完全重定向的文件中可以找到这两个函数的入口（可以使用 **objdump 工具反汇编.elf 文件查找这两个函数的入口**），记为 0xP 和 0xQ。则在 testbench 当中检查当前周期退休指令的 PC 值，当 PC 值为 0xP 时，表明发生了 guard_start 调用，并认为此时为分类计数的起点。当 PC 值为 0xQ 时，表明发生了 guard_end 调用，并认为此时为分类计数器的终点。使用这种方法，就可以较为精确地统计源码中某个代码段的指令执行情况。应当注意的是，对于 gurad_start 和 guard_end 这种非常简单的函数调用，GCC 编译器极有可能对其进行 inline 优化，导致运行时根本不会跳转到这两个函数的入口执行。为此，**需要在 ./lib/Makefile 中对 CFLAGS 参数进行修改，添加 -fno-inline 和 -g 两个选项**，以保证主函数对这两个函数的调用以程序跳转的方式进行。

此外，编译器对于 guard_start 和 guard_end 的链接后地址分配因实际情况而在不同环境下有所不同，因此**用户应当自行核对这两个函数的链接后地址，并修改 ./tb/tb.v 的相关宏定义，以使得计数器正常工作**。需要核对的内容如下（核对方法详见第 5.2 节）：

```
`define GUARD_START_PC    40'h1850
`defien GUART_END_PC      40'h1910
```

完成上述所有修改后, 在 testbench 中定义的指令统计计数器应当可以正常工作并输出结果。在成功仿真了 case/conv_test/conv_test.c 后, workdir 目录下包含以下 3 个相关的输出文件:

- a) run.log 文件记录了 CPI、cache 缺失率和分支预测统计信息。
- b) inst_count 文件按照 opcode 的分类对每种类型指令的执行次数进行了计数。这个文件的格式示例为:

```
op[1:0]=11:
00: xx
01: xx
...
1f: xx

op[1:0]=00:
0: xx
1: xx
...
7: xx

...etc.
```

其中“op[1:0]=”行表示所统计指令的最低 2 位。如果 op[1:0] 为 11 则为 32 位指令, 那么接下来每一行的开头 2 个字符以 **16 进制**表示了 op[6:2]这 5 位, 它与 op[1:0]共同构成了 7 位的 opcode。如果 op[1:0] 为 10、01、00 则为 16 位指令, 那么接下来的每一行开头的字符以 **16 进制**表示了指令编码最高 3 位 (即[15:13]) 的值, 它作为指令类型的扩展与 op[1:0]共同指示了指令所属的类型。

每行冒号后的数值 xx 则以 **10 进制**统计了该指令的执行数量。

利用 inst_count 文件即可进行指令类型的统计。在实验文件夹下提供了一个可执行文件 inst_proc, 用户可以**把 inst_count 和 inst_proc 放在同一路径下, 然后运行 inst_proc**, 即可得到一个按照较粗的粒度分好类别的统计文件 inst_class, 可以直接按照这个文件的结果来进行绘图。

如果感兴趣的话, 也可以在阅读 RISC-V 的 spec 文件后, 自行分析 inst_count 的内容, 并作更详细的分类统计。

- c) inst_record 文件详细记录了程序运行中的每一条退休指令的 PC 值, 这个文件比较长, 感兴趣的用户可以查阅这个文件来分析源码的详细运行情况。

五、 实验步骤

5.1 核对待改动的文件

实验所用的全部材料已经放在 EDA 集群上的/home/ECDesign/ECDesign_share/lab9/文件夹下。由于本实验涉及比较多的文件修改, 在开始仿真前务必仔细核对以下的每一个文件是

否已经进行了相应替换或者修改（假设当前正处在目录 smart9_release 下）：

- a) 将 lab9 文件夹下提供的 **axi_slave128_copy.v** 复制到目录 **rtl/platform/amba/axi/** 下。这个新的总线模块将其内部的内存模型映射到基地址 40'h0200_0000，用于仿真时卷积层输入数据的初始化。参见第 3.2 节。
- b) 用 lab9 文件夹下提供的 **soc.v** 替换原有的 **rtl/platform/common/soc.v** 文件。该文件对 axi_slave128_copy 进行了实例化，并提供上层互连。参见第 3.2 节。
- c) 用 lab9 文件夹下提供的 **Makefile** 替换原有的 **lib/Makefile** 文件。新的 Makefile 在 CFLAGS 中加入了 -fno-inline 和 -g 两个选项，避免 guard_start 和 guard_end 被内联优化。参见第 4.2 节。
- d) 用 lab9 文件夹下提供的 **tb.v** 替换原有的 **tb/tb.v** 文件。新的 testbench 加入了对不同类型指令在仿真时的执行数量的统计。参见第 4.2 节。
- e) 检查 **lib/crt0.s** 中是否对 **PMU 相关寄存器进行了初始化**。若没有，可以用 lab9 文件夹下的 crt0.s 替换原有的 lib/crt0.s 文件。正常情况下，已经完成了实验 6-8 的 Smart 平台不需要执行该步骤。
- f) 检查 **lib/clib/pmu.h** 文件中所有内联汇编语句 **asm** 后面是否有添加 **volatile** 关键字。若没有，可以用 lab9 文件夹下的 pmu.h 替换原有的 lib/clib/pmu.h 文件。
- g) 将 lab9 文件夹下的整个 **conv_test** 文件夹复制到用户的 **SMART 目录 case/** 下。文件夹中应当包含 conv_test.c、conv_test.h 和 work.h 三个源码文件，以及输入数据文件 data_in、辅助统计的二进制文件 inst_proc。

举例而言，假设用户当前已经处在自己的 smart9_release 工作目录下，可以通过执行以下命令来完成文件替换：

```
cp -rf /home/ECDesign/ECDesign_share/lab9/ ~
cp ~/lab9/axi_slave128_copy.v ./rtl/platform/amba/axi/
cp ~/lab9/soc.v ./rtl/platform/common/
cp ~/lab9/Makefile ./lib/
cp ~/lab9/tb.v ./tb/
cp ~/lab9/crt0.s ./lib/
cp ~/lab9/pmu.h ./lib/clib/
cp -rf ~/lab9/conv_test/ ./case/
```

执行上述命令时，所有 overwrite 提示均选择 y。

5.2 执行仿真

完成上述核对后，可以正式开始仿真。

务必注意，由于新提供的 tb.v 文件是针对单核心的情况进行指令分布统计的，因此配置 C910 核心时“Core Num”选项一定要设置为 1，即仅使用一个核心。如果不符，应当自行重新生成一个核心，参见第 4.1 节。

在 ./workdir 目录下运行以下命令即可进行仿真：

```
../tools/run_case ../case/conv_test/conv_test.c
```

视编译环境和代码的不同，工具链生成的完成重定向二进制文件中，函数 guard_start 和 guard_end 的函数入口可能与本文档提供的值不同，每次仿真时建议进行一次核对。提交仿

真后，在目录 case/conv_test/下对生成的二进制文件执行以下命令可以进行反汇编：

```
../../tools/toolchain/RV64GC/bin/riscv64-unknown-elf-objdump -S
conv_test.elf > conv_test.s
```

然后查看 conv_test.s 的内容，找到 guard_start 和 guard_end 的重定向入口，例如：

```
void guard_start()
{
    1850:      1141      addi    sp,sp,-16
    1852:      e406      sd      ra,8(sp)
```

及：

```
void guard_end()
{
    1910:      1141      addi    sp,sp,-16
    1912:      e406      sd      ra,8(sp)
```

则在该例子中，guard_start 的入口地址为 40'h1850，而 gurad_end 的入口地址为 40'h1910。此时核对 tb/tb.v 中开头部分的如下定义：

```
`define GUARD_START_PC    40'h1850
`define GUART_END_PC      40'h1910
```

如果与 conv_test.s 中的结果不匹配，应当修改 tb/tb.v 的这两行宏定义，终止已经提交的仿真任务，删除 conv_test.s 文件，然后重新执行仿真。关于这两个函数在本实验中的作用，参见第 4.2 节。

5.3 分析仿真结果

考虑到完整卷积层的输入规模较大，在 SMART 上仿真需要的时间很长。本实验截取了一组较小规模的数据，所使用的卷积函数输入规模为：

```
para.input_w = para.input_h = 19
para.input_c = 32
para.kernel_size = 3
para.kernel_n = 64
para.pad = 1
para.stride = 2
```

仿真结束后，利用 workdir 下生成的以下文件来进行结果分析：

1. CPI、Cache 缺失率和分支预测统计

利用 run.log 文件统计 CPI、Cache 缺失率和分支预测数据，与实验 7-8 类似。将仿真结果汇总成下表所示的形式：

configure	all prediction off	BPE on, BTB off	all prediction on
cycle			
insts			
CPI			
conditional branch miss			

L1_Dread access			
L1_Dread miss			
L1_Dread miss_rate			
L1_Dwrite access			
L1_Dwrite miss			
L1_Dwrite miss_rate			
L2_Dread access			
L2_Dread miss			
L2_Dread miss_rate			
L2_Dwrite access			
L2_Dwrite miss			
L2_Dwrite miss_rate			

其中，分支预测器需要统计：所有预测关闭、仅开启 BPE（关闭 BTB）、所有预测开启三种情况，观察 CPI 和条件跳转 miss 数量的变化，因此仿真需要进行三次，修改分支预测开关的方法与实验 8 相同。而由于 Cache 大小的变化对实验结果的影响非常小，所以选定任意一种 Cache 配置均可，不需要重复仿真。

2. 指令分布统计

仿真结束后，在 workdir 下会生成数个统计文件，相关说明详见第 4.2 节末尾。要求利用 inst_count 文件的结果进行不同类型指令的执行情况统计，并绘制成统计图。本实验已经事先提供了一个可执行文件 inst_proc，直接在 workdir 目录下执行：

```
./inst_proc
```

即可生成一个粗粒度的指令分布统计 inst_class 文件，用户可以直接利用 inst_class 来作为统计结果并绘图。**指令分布选取任意一次的仿真结果做一次统计即可。**

注：使用 vi 打开 inst_class 文件时，如果遇到中文乱码，可以在末行模式（命令模式下用冒号:进入）下输入 set encoding=utf-8 来解决。

(optional)感兴趣的同学也可以查阅 RISC-V spec 文件后自行对 inst_count 文件进行分析，inst_count 文件内容的解释参见第 4.2 节。此外，workdir 下还提供一个文件 inst_record，详细记录了程序运行中的每一条退休指令的 PC 值。该文件较长，感兴趣的同学可以查阅这个文件来分析源码的详细运行情况。

六、 实验报告模板

YOLO 卷积函数的 SMART 仿真

1、实验目的

通过将 YOLO 源码的卷积函数放在 SMART 上进行仿真和性能监测，进一步介绍嵌入式 C 裸机程序在 C910 上的开发过程和仿真原理。通过观测分支预测开关对程序性能的影响，理解分支预测在现代处理器中的重要性。通过指令分布统计，了解内存墙的概念，并进一步理解缓存存在现代处理器中的重要性。

2、实验步骤（包括实验结果，数据记录、截图等）

- 1) 文件核对和执行仿真
- 2) CPI、Cache 缺失率和分支预测统计
- 3) 指令分布统计绘图

3、实验分析和总结

4、实验收获、存在问题、改进措施或建议等